

Código como arte, arte como código: el lenguaje de programación SCuLPT

Seudónimo: [SEUDÓNIMO]

Resumen

Todo el mundo debería tener la oportunidad de programar igual que lo hace cualquier programador, sin importar la condición física, cognitiva o social. El lenguaje de programación SCuLPT (*Simple Cubic Language for Programming Tasks*) es un lenguaje de programación y ecosistema artístico que pretende retar las convenciones de los lenguajes de programación tradicionales y las nociones sobre el pensamiento computacional. SCuLPT es un lenguaje de programación visual, tangible y modular el cual disrumpe la interfaz estandar de teclado y pantalla, mediante una interfaz física que se puede tocar, manipular y compartir junto a su especificación técnica, código fuente y documentación. Esto se realiza mediante la construcción de constructos artísticos los cuales se pueden ensamblar para crear esculturas que generan programas computables. Junto al lenguaje, se presenta SCuLPTER para crear y ejecutar programas antes de ensamblarlos en SCuLPT. Adicionalmente, cuenta con documentación sobre su uso y desarrollo y tutoriales para aprender a programar con SCuLPT. El lenguaje construido es totalmente capaz y Turing-completo el cual sirve como una herramienta creativa tambien. Usando estas herramientas, se realiza una validación empirica de la experiencia con novatos y expertos en programación. Los resultados muestran ligera proeza computacional al resolver problemas de programación entre las poblaciones y unas piezas de arte interesantes.

Palabras clave: pensamiento computacional, lenguajes de programación visuales, diseño inclusivo, computación creativa.

1. Introducción

Desde el nacimiento de la programación, la interfaz a través de la cual los programas se materializan en código ha sido esencialmente la misma: un teclado y una pantalla. Llamamos a este arreglo el *paradigma de la máquina de escribir*. Es una herencia tan casual que rara vez se cuestiona, pero sus consecuencias son profundas y de a ratos silenciosas. Condiciona quiénes pueden programar, cómo aprenden a hacerlo y qué formas de expresión consideramos correctas dentro de la computación.

Tres problemas convergen en este punto. Primero, la interfaz teclado-pantalla discrimina contra usuarios cuyas capacidades físicas, cognitivas o sensoriales difieren del promedio asumido por sus diseñadores [9]. Segundo, la programación impone una curva de aprendizaje pronunciada y un umbral de entrada alto [3], lo que desalienta a una parte importante de la población incluso antes de que llegue a escribir su primera línea de código. Tercero, los lenguajes de programación contemporáneos y su docencia descansan sobre esta interfaz plana, trayendo consigo los dos problemas anteriores y sus repercusiones sobre quienes pretenden programar y cómo se acercan a la programación. Si la intención es que la programación sea un medio universal y abierto para el pensamiento, entonces el paradigma de la máquina de escribir es un obstáculo para pensar de formas distintas de lograrlo. La solución podría radicar en la intersección entre la computación y el arte [22].

La pregunta que orienta este trabajo es si es posible diseñar un lenguaje de programación que (1) materialice los programas en un soporte radicalmente distinto al texto plano, (2) sirva y preserve las funcionalidades computacionales y las propiedades formales que se esperan de un

lenguaje y (3) brinde un ecosistema de herramientas creativas que permitan al usuario explorar e interactuar con la computación de forma libre y expresiva. La hipótesis es entonces que la expresión estética puede servir como vehículo legítimo para la computación. Ensamblar piezas tangibles puede ser, al tiempo, un acto escultórico y un acto de computación.

SCuLPT (*Simple Cubic Language for Programming Tasks*) es la implementación de esa hipótesis. En términos generales, SCuLPT presenta (1) un lenguaje de programación Turing-completo formalmente especificado, (2) una colección de piezas y sus características de construcción y (3) SCuLPTER, un entorno y *runtime* web escrito en Scala 3 y Scala.js que permite escribir, depurar y ejecutar programas antes de ensamblarlos físicamente.

Este artículo presenta el diseño físico y lógico de SCuLPT, su semántica, una demostración formal condensada de su Turing-completitud y una validación empírica con varios grupos de participantes. La contribución principal no es técnica en sentido estricto, sino conceptual: mostrar que la *tangibilidad*, la *libertad estética* y la *computación* pueden coexistir en un mismo lenguaje sin que ninguna de las tres se sacrifique. SCuLPT muestra que el lenguaje puede ser usado efectivamente para hacer computación sin requerir conocimientos previos en computación, puede ser utilizado para construir programas computacionales por expertos, y puede ser usado como un medio de expresión artístico.

2. Planteamiento del problema

La programación, tal como se enseña y se practica, excluye a quien no encaja en su molde dominante. Esto se descompone en tres tensiones interrelacionadas que conviene examinar por separado.

Accesibilidad de la interfaz. El teclado y la pantalla son artefactos históricamente situados: fueron diseñados para usuarios con destreza manual binaural, visión clásica y motricidad fina sostenida. Cualquier desviación de ese perfil como la pérdida temporal o permanente de movilidad en un brazo, dislexia, baja visión, edad temprana o neurodivergencia convierte la interfaz en una barrera activa [9, 12]. Mientras que disciplinas como la música o la pintura han desarrollado durante siglos múltiples modos de acceso, la programación se ha aferrado a un único canal el cual, aunque brinda soluciones de accesibilidad, no es intrínsecamente inclusivo.

Umbral de entrada cognitivo. Aprender a programar requiere dominar una sintaxis exótica, una semántica abstracta y un modelo mental de ejecución que el principiante no puede inspeccionar directamente. Estudios sistemáticos del proceso de aprendizaje [3, 16] documentan patrones de dificultad consistentes a lo largo de instituciones distintas. La consecuencia es un *filtro* que opera antes de que el aprendiz pueda evaluar si la programación le interesa.

Monotonía paradigmática. La mayor parte de los lenguajes actuales descienden, con variaciones de superficie, de un mismo linaje sintáctico. Las propuestas que desafiaron este precepto como lenguajes visuales, en bloques o end-user programming mejoraron la accesibilidad textual pero, en su gran mayoría, conservaron el supuesto fundamental de que el programa se inscribe en una superficie bidimensional plana. Llevar la programación al espacio tridimensional, al tacto y a la manipulación de objetos sigue siendo territorio poco explorado, pese a evidencia robusta sobre los beneficios del aprendizaje manipulativo en otras disciplinas [10, 11].

Pregunta de investigación. A partir de las tres tensiones anteriores se formula la pregunta guía: *¿es posible diseñar un lenguaje de programación tangible, Turing-completo y estéticamente abierto que funcione como puerta de entrada a la computación para audiencias más diversas, sin sacrificar el rigor formal del que depende cualquier lenguaje convencional?* El resto del artículo desarrolla una respuesta afirmativa, teniendo en cuenta distintas perspectivas y alternativas.

3. Trabajo relacionado

SCuLPT se inserta en una conversación que ya tiene varias décadas sobre cómo abrir la programación a públicos no especializados. Esta ha producido tres familias de lenguajes que conviene revisar por separado, ya que cada una resuelve parcialmente el problema que SCuLPT plantea. Se nombran a estas familias *lenguajes visuales y de bloques*, *lenguajes tangibles para la enseñanza de la computación* y *computación como práctica artística (creative computing)*. Al cierre de la sección se posiciona a SCuLPT frente a las tres.

3.1. Lenguajes visuales y de bloques

La línea de los lenguajes basados en bloques arranca con *LogoBlocks* [2] en 1996 y se consolida con *Scratch* [15], desarrollado en el *Lifelong Kindergarten Group* del MIT Media Lab. Scratch reemplaza la sintaxis textual por bloques arrastrables cuya forma define qué encaja con qué. Esto abre una exploración interesante sobre la sintaxis. La sintaxis en este tipo de lenguajes adquiere una dimensión geométrica. Los conjuntos de reglas que gobiernan la construcción de programas son ahora reglas físicas. Esa idea ha sido extendida por *Blockly* [5], una biblioteca de Google que genera código en lenguajes objetivo (JavaScript, Python, Dart) a partir de programas en bloques, y por *Snap!* [6], una reimplementación de Scratch con funciones de primer orden y bloques personalizables, orientada a la educación universitaria.

Estas propuestas pretenden reducir drásticamente la fricción sintáctica para los principiantes y son hoy la herramienta de elección en la enseñanza temprana de la computación. Sus limitaciones, sin embargo, son congruentes con el diagnóstico de la problemática. Los bloques viven en una pantalla, dependen de un teclado (usualmente) y un *mouse*, y, por diseño, no admiten libertad estética significativa más allá de cambiar colores de fondo. La interfaz se ha simplificado, la computación mejorado, pero su soporte material no ha cambiado.

3.2. Lenguajes tangibles para la enseñanza

Una segunda familia traslada la programación al mundo físico. *AlgoBlock* [20] fue uno de los primeros sistemas en proponer bloques físicos conectables como medio para construir programas, principalmente con propósitos colaborativos entre niños. *Tern* [7] extendió la idea con piezas de madera reconocidas por visión por computador, sin necesidad de electrónica embebida. Más recientemente, *KIBO* [19] combina bloques de madera con códigos de barras leídos por un robot móvil, y *Cubetto* [13], comercializado en múltiples mercados, usa una cuadrícula y fichas magnéticas para programar el desplazamiento de un pequeño robot. *Osmo Coding* [21] sigue una línea híbrida, las piezas son físicas pero la ejecución ocurre en una tableta que las observa por la cámara.

Estos sistemas demuestran ampliamente que la tangibilidad reduce barreras cognitivas y motivacionales en públicos estudiantiles, en línea con literatura previa sobre *tangible user interfaces* [8, 10]. Comparten, no obstante, tres limitaciones recurrentes. Primero, todos están diseñados como *herramientas educativas*: su poder expresivo es intencionalmente acotado y aplicado a un dominio específico, usualmente robótica. Esto brinda una experiencia de programación más concreta, manipulativa y estimulante para los estudiantes, pero no dejan de ser entonces lenguajes de dominio específico, o en otras palabras, herramientas didácticas programables. Segundo, las piezas son cerradas. Vienen en una forma, un color y un tamaño preestablecidos por el fabricante, y el usuario no puede siempre modificarlas sin destruir o alterar su funcionalidad. Esta libertad, tanto estética como funcional, por diseño está ausente. Este afán por controlar la experiencia del usuario es comprensible en un contexto educativo, sin embargo, limita la expresividad y la cantidad de usuarios que pueden usar estos sistemas. Libertad de modificación con las piezas es relevante para usuarios con necesidades de accesibilidad, por ejemplo, que pueden necesitar piezas de un tamaño, material o morfología particular para manipularlas. Tercero, la mayoría

de estos sistemas vienen con una etiqueta de precio que no todas las instituciones educativas o individuos pueden pagar. Adicional a necesitar hardware (y a veces software) especializado, el costo de las piezas, su mantenimiento y acceso puede ser un obstáculo para su adopción. Esto es especialmente relevante en contextos de bajos recursos, donde la accesibilidad a herramientas educativas resulta crucial para cerrar brechas de aprendizaje.

3.3. Computación como práctica artística

Una tercera tradición aborda el problema desde un ángulo distinto. Mantener la interfaz textual de la programación pero dirigida explícitamente a la producción artística. *Processing* [14], ofrece un dialecto de Java pensado para artistas visuales y diseñadores, y popularizó la idea de que escribir código es una forma legítima de práctica artística. *Sonic Pi* [1], construye sobre el motor de síntesis SuperCollider un superconjunto de Ruby orientado a *live coding* musical, originalmente concebido como herramienta educativa para escuelas en Inglaterra. La práctica del *live coding*, tanto musical como visual, ha sido sistematizada como movimiento artístico-computacional propio [4], con festivales y comunidades dedicadas (e.g., *Algorave*).

Estas iniciativas han demostrado que la programación *puede* ser arte y que existe audiencia entrenada y entusiasta para esa visión. Sin embargo, comparten un límite estructural con la primera familia. No abandonan el paradigma de la máquina de escribir. El artista que hace *live coding* sigue tecleando sobre una pantalla en vivo (y estresándose porque el código no compila) la producción artística final es sonido, video o imagen, no el cuerpo físico del programa.

3.4. Posicionamiento de SCuLPT

SCuLPT se sitúa en la intersección poco transitada de las tres familias anteriores. Comparte con los lenguajes de bloques la idea de que la sintaxis puede ser una propiedad geométrica de las piezas esencial para la construcción de programas. La intuitividad natural de encajar piezas es un recurso poderoso para asimilar conceptos sintácticos complejos y los aterriza a una experiencia y lógica de construcción concreta y entendible. Encaja o no encaja. Comparte con los lenguajes tangibles la apuesta por sacar la programación de la pantalla y llevarla al espacio tridimensional. Habilitarle al mundo a programar implica necesariamente abrir la programación a formas de acceso distintas a la tradicional. Se pretende aprovechar la naturaleza manipulativa y abierta de las fichas para abrir la programación a usuarios con distintas capacidades, recursos y necesidades de accesibilidad. Finalmente, comparte con la tradición del *creative computing* la convicción de que la práctica artística es un vehículo legítimo para la computación, no un adorno posterior. La Tabla 1 resume las diferencias.

Tabla 1: Comparación de SCuLPT con familias relacionadas.

	Bloques/visuales	Tangibles edu.	Creative comp.	SCuLPT
Soporte físico	No	Sí	No	Sí
Turing-completo	Parcial	No	Sí	Sí
Libertad estética	Baja	Baja	Media	Muy Alta
Público objetivo	Estudiantes/Novatos	Niños	Artistas	Mixto

Hasta donde se sabe, SCuLPT es el primer lenguaje que combina *tangibilidad física*, *Turing-completitud demostrada formalmente* y *libertad estética radical* en un mismo diseño. La radicalidad de la apertura estética merece subrayarse: en SCuLPT el usuario puede modificar libremente forma, tamaño, color, material y textura de las piezas; lo único restringido es la topología de las conexiones. Una pieza puede ser un cubo PLA estándar o una figura animal tallada en madera, y el programa funcionará igual mientras los puntos de acople respeten la especificación.

4. Metodología

4.1. Diseño del lenguaje

SCuLPT es, en rigor, dos lenguajes acoplados. El primero es un *lenguaje visual y físico*: una especificación de los componentes que pueden existir y de las reglas de construcción entre ellos. El segundo es un *lenguaje de programación* que define el comportamiento que cada componente representa cuando se ensambla junto a otros. Cada programa SCuLPT es, simultáneamente, un objeto construible en el espacio tridimensional y una expresión computable.

El diseño del lenguaje sigue un enfoque centrado en el usuario, con el objetivo explícito de que el conjunto de componentes fuera reconocible para audiencias diversas. Cada componente se distingue mediante una forma simple pero única, de modo que pueda identificarse sin recurrir a conocimiento previo ni a la lectura de texto. La sintaxis del lenguaje es una propiedad geométrica: dos piezas encajan o no encajan, y la legalidad del programa se inspecciona física y mecánicamente.

El conjunto de operaciones es deliberadamente mínimo. SCuLPT tiene solo 13 instrucciones que operan sobre pilas LIFO de números racionales. Las pilas son la única estructura de datos del lenguaje, y se crean implícitamente la primera vez que una instrucción las referencia. Cualquier forma no reservada para una instrucción puede designar una pila, lo que extiende la libertad estética del lenguaje también al espacio de datos. Sobre este conjunto mínimo se obtiene Turing-completitud, como se demuestra en la Sección 4.5.

La estética de los componentes es, por diseño, abierta. La especificación solo restringe las conexiones entre piezas y el conjunto de *features* que cada bloque debe ofrecer; la forma final, el tamaño, el material, el color y la textura quedan al arbitrio de quien construye el programa. Un cubo PLA estándar, una figura tallada en madera o un trozo de papel garabateado y mal doblado pueden ser piezas válidas mientras los puntos de acople respeten la especificación. Esta apertura es lo que permite, al mismo tiempo, ajustar las piezas a necesidades de accesibilidad particulares (tamaño, peso, contraste, textura) y abrir el lenguaje como medio de expresión artística.

4.2. Implementación física

La implementación de referencia utilizada en las pruebas consta de piezas pequeñas impresas en ácido poliláctico (PLA) de colores variados y piezas de melamina blanca, conectadas mediante imanes y juntas roscadas tipo codo. Esta combinación permite el ensamblaje y desensamblaje repetido sin herramientas, ofrece resistencia estructural suficiente para manipular el programa sin que colapse y facilita modificarlo durante la sesión. El conjunto de piezas se distribuye como modelos imprimibles y planos de corte, de modo que la implementación de referencia pueda replicarse, adaptarse a otros materiales o ajustarse a necesidades particulares de accesibilidad.

La fisicalidad del lenguaje conlleva un compromiso. Al ser un medio estrictamente físico, la ejecución de un programa SCuLPT depende de un *intérprete humano*, que recorre las piezas y aplica las instrucciones. La interpretación humana es parte del ejercicio de SCuLPT como acto performativo, pero no es confiable. Los errores son inevitables y el lenguaje a nivel físico no es tolerante a fallos de interpretación. Este compromiso motiva la existencia de un intérprete software complementario.

4.3. El *runtime* SCuLPTER

Para compensar la fragilidad de la interpretación humana, el ecosistema incluye SCuLPTER (*Simple Cubic Language for Programming Task's Environment and Runtime*), un intérprete de referencia escrito en Scala 3 y compilado a JavaScript mediante Scala.js. SCuLPTER opera sobre la notación textual del lenguaje, equivalente a la notación física. Expone un editor donde se escribe el programa, controles separados para lexar, parsear y ejecutar, y un visor de estado paso a paso que muestra el contenido de cada pila en cada momento de la ejecución. Esta vista por pasos lo hace también útil como herramienta de depuración antes del ensamblaje físico.

Por correr en el navegador no requiere instalación, lo que lo hace distribuible inmediatamente. Adicionalmente, el sitio cuenta con documentación sobre el lenguaje, su uso y desarrollo, y una galería de obras artísticas hechas con SCuLPT. El código fuente del lenguaje y del intérprete están disponibles bajo licencia abierta [17, 18].

4.4. Especificación del lenguaje

SCuLPT como lenguaje de programación se inspiró en lenguajes de ensamblador, con un conjunto de instrucciones mínimo y una semántica de bajo nivel que opera sobre pilas de números racionales. Su único tipo de dato son los números racionales. Las pilas se crean en la primera referencia, no tienen cota superior de tamaño. SCuLPT no tiene variables ni funciones, pero se pueden simular con pilas y bucles. Su ejecución es secuencial, no concurrente y completamente dependiente de un interprete humano o SCuLPTER. La Tabla 2 resume la semántica de las instrucciones operativas.

Tabla 2: Instrucciones operativas de SCuLPT. S designa una pila; n un número literal.

Instrucción	Semántica
push n S	Apila n en S .
pop S	Retira el tope de S . Es <i>no-op</i> si S está vacía.
mov S_1 S_2	Saca el tope de S_2 y lo apila en S_1 . Si S_2 está vacía, apila nil en S_1 .
dup S	Duplica el tope de S . Si S está vacía, apila nil .
cmp S	Saca los dos topes a, b y apila 0 si $a = b$, -1 si $a < b$, 1 si $a > b$. Si algún operando es nil o la pila tiene menos de dos elementos, apila nil .
cmp S n	Reemplaza el tope a por 0, -1 o 1 comparando con n . Si a es nil o S está vacía, apila nil .
? S	Saca el tope; si es ≥ 0 ejecuta la siguiente instrucción, si es < 0 la salta.
jmp k	Desplaza el contador de programa en k posiciones (puede ser negativo).
neg S	Niega el tope de S <i>in situ</i> .
mul S	Saca los dos topes y apila su producto.

SCuLPT soporta seis operaciones aritméticas, suma, resta, multiplicación, división, módulo y comparación, en dos variantes físicamente distintas: *unaria* (opera sobre los dos primeros elementos desde el tope de una pila) y *binaria* (opera sobre el tope y un literal). Para operaciones no conmutativas, el primer operando es el lado izquierdo y el segundo el derecho; **sub stack 3** calcula *tope_de_pila* $- 3$. En aritmética indefinida (e.g., división por cero) se apila un valor *nil*; encontrarlo en una operación distinta de la comparación detiene el programa en estado de error. El control de flujo se construye con bucles físicos (conexiones cíclicas de piezas), saltos relativos y un *bloque pregunta* que ramifica según el signo del tope de una pila.

4.5. Turing-completitud

SCuLPT es Turing-completo. La demostración procede por construcción de una traducción Φ que mapea una máquina de Turing arbitraria $M = (Q, \Gamma, b, \Sigma, \delta, q_0, F)$ a un programa de SCuLPT $\Phi(M)$ que la simula paso a paso. La cinta se codifica con dos pilas, **left** y **right**: el tope de **right** representa la celda bajo el cabezal y el tope de **left** la celda inmediatamente a su izquierda. Los símbolos no blancos se codifican como enteros positivos, y el blanco se identifica con el valor especial **nil**, esto es, $b \mapsto \text{nil}$. Una pila auxiliar **aux** soporta la comparación no destructiva durante las transiciones.

Cada estado no terminal $q \in Q$ se traduce a un bloque $B(q)$ de tamaño constante $|B(q)| = 2 + 12|\Gamma|$ instrucciones, dividido en tres fases. *Fase 1*: lee la celda actual y copia su valor a **aux** (2 instrucciones); *Fase 2*: es una cadena de despacho que prueba el valor contra cada símbolo $\sigma_i \in \Gamma$

usando `cmp`, `mul`, `neg` y `?` ($7|\Gamma|$ instrucciones); *Fase 3*: ejecuta el manejador correspondiente a la transición $\delta(q, \sigma_i) = (q', \sigma'_i, D)$, que escribe el nuevo símbolo y mueve el cabezal ($5|\Gamma|$ instrucciones). Los estados de aceptación se traducen al bloque `HALT`, que termina la ejecución.

Un lema clave de la construcción muestra que *no se requiere código explícito para extender la cinta hacia el infinito en blanco*. El argumento descansa en dos hechos: (i) las operaciones `dup`, `mov` y `cmp` sobre pilas vacías o con operandos faltantes producen `nil`; (ii) la codificación identifica el símbolo blanco con ese mismo valor, i.e., $\text{enc}(b) = \text{nil}$. Como el operador `?` trata `nil` como rama “falsa” y `nil` se propaga a través de `cmp`, `mul` y `neg`, las pilas vacías representan de forma natural una cola infinita de celdas en blanco sin necesidad de materializarlas. Así, cada $B(q)$ tiene tamaño fijo independientemente de la frontera de la cinta.

Sea Π_k la configuración de SCuLPT después de ejecutar k pasos. Decimos que Π_k es *k-consistente con M* si `aux` está vacía, el tope de `right` codifica la celda actual, los demás elementos codifican el resto de la cinta a ambos lados del cabezal y el contador de programa apunta al inicio de $B(q_k)$. Por inducción sobre k se prueba que la k -consistencia se preserva, lo que da los dos teoremas siguientes:

Teorema 1 (Preservación del halting). *Dada una máquina de Turing M termina sobre la entrada w en n pasos, entonces $\Phi(M)$ alcanza el bloque HALT y termina.*

Teorema 2 (Preservación de la no-terminación). *Dada una máquina de Turing M que no termina sobre la entrada w, entonces $\Phi(M)$ no alcanza el bloque HALT y no termina.*

La traducción Φ es total, computable y produce programas de tamaño $|\Phi(M)| = O(|Q| \cdot |\Gamma|)$. Como toda función computable se calcula con alguna máquina de Turing, se sigue:

Teorema 3. *SCuLPT es Turing-completo.* □

Como verificación empírica de la traducción se implementó el juego de *Busy Beaver* de tres estados (BB_3) en SCuLPT, que escribe seis 1's en la cinta antes de detenerse al cabo de 14 pasos. El desarrollo completo de la prueba, las definiciones precisas de los *offsets* de salto y el código del BB_3 se encuentran en el documento de tesis original.

4.6. SCuLPT en acción



Figura 1: Programa Fibonacci en SCuLPT: las piezas distintas designan operaciones y las flechas el orden de ejecución. El programa llena la pila “círculo” con la secuencia de Fibonacci.

La Figura 1 muestra un programa que calcula la secuencia de Fibonacci. La misma estructura se expresa en la notación textual del intérprete SCuLPTER, que omite las flechas de conexión y se lee como un lenguaje ensamblador.

El programa coordina cuatro pilas con roles diferenciados. **circle** acumula la salida, **star** y **triangle** funcionan como registros de los dos términos previos F_{n-2} y F_{n-1} ; **hexagon** es una pila aritmética temporal donde se calcula el nuevo término. La inicialización (`push 0 star`, `push 0 circle`, `push 1 circle`, `push 1 triangle`) instala el invariante de arranque, con F_0 en **star**, F_1 en **triangle** y la secuencia ya iniciada en **circle**. A partir de ahí, cada vuelta del bucle (`dup` $\rightarrow$ `mov` $\rightarrow$ `add` $\rightarrow$ `mov`) duplica los dos términos previos, los traslada a **hexagon**, suma allí, copia el resultado a **circle** y rota los registros para la siguiente iteración. La instrucción `jmp -10` cierra el bucle.

La Tabla 3 traza la ejecución instrucción por instrucción. Cada fila muestra el estado de las cuatro pilas *después* de ejecutar la instrucción, con el tope a la derecha. Las primeras cuatro filas corresponden a la inicialización, las once siguientes a la primera vuelta del bucle, que produce $F_2 = 1$; la última fila resume el estado tras la segunda vuelta, que añade $F_3 = 2$ a **circle**.

Tabla 3: Traza de ejecución del programa Fibonacci. Notación de pila: `[]` pila vacía; en `[a, b, c]` el tope es `c`. Cada fila muestra el estado tras ejecutar la instrucción.

#	Instrucción	star	triangle	hexagon	circle
1	<code>push 0 star</code>	[0]	[]	[]	[]
2	<code>push 0 circle</code>	[0]	[]	[]	[0]
3	<code>push 1 circle</code>	[0]	[]	[]	[0, 1]
4	<code>push 1 triangle</code>	[0]	[1]	[]	[0, 1]
5	<code>dup triangle</code>	[0]	[1, 1]	[]	[0, 1]
6	<code>dup star</code>	[0, 0]	[1, 1]	[]	[0, 1]
7	<code>mov hexagon star</code>	[0]	[1, 1]	[0]	[0, 1]
8	<code>mov hexagon triangle</code>	[0]	[1]	[0, 1]	[0, 1]
9	<code>add hexagon</code>	[0]	[1]	[1]	[0, 1]
10	<code>pop star</code>	[]	[1]	[1]	[0, 1]
11	<code>mov star triangle</code>	[1]	[]	[1]	[0, 1]
12	<code>dup hexagon</code>	[1]	[]	[1, 1]	[0, 1]
13	<code>mov circle hexagon</code>	[1]	[]	[1]	[0, 1, 1]
14	<code>mov triangle hexagon</code>	[1]	[1]	[]	[0, 1, 1]
15	<code>jmp -10</code>	[1]	[1]	[]	[0, 1, 1]
<i>(segunda vuelta, resumida)</i>		[1]	[2]	[]	[0, 1, 1, 2]

La traza vuelve visible la semántica LIFO del lenguaje. Cada `mov` y cada operación aritmética actúan exclusivamente sobre el tope de la pila implicada, y todo movimiento de datos pasa por esa frontera. Cada fila es, además, lo que un intérprete humano sostiene en sus manos tras manipular físicamente la pieza correspondiente; SCuLPTer reproduce exactamente esta vista paso a paso en el navegador, lo que permite depurar el programa antes del ensamblaje. La notación textual, sin embargo, no representa los bucles físicos directamente y obliga a expresarlos como saltos relativos: el `jmp -10` de la fila 15 cierra el bucle que en la Figura 1 aparece materializado como una conexión cíclica entre piezas.

5. Resultados

5.1. Protocolo de validación

La validación empírica se realizó en seis sesiones con caracterizados en distintos perfiles: novatos sin experiencia previa en programación, programadores expertos con años de experiencia, estudiantes de secundaria, niños y profesionales del campo artístico. La diversidad de perfiles fue intencional. SCuLPT aspira a ser inclusivo, y la validación debe ponerlo a prueba con preámbulos cognitivos y motivacionales muy distintos. El objetivo de estas sesiones de validación, con respecto a los perfiles de los usuarios es: (1) mostrar la posibilidad de SCuLPT como un lenguaje de programación que rompe las barreras de accesibilidad para nuevos usuarios. (2) La efectividad de

SCuLPT como lenguaje de programación, permitiendo a programadores con experiencia construir programas completos. (3) El uso de la programación como un medio de expresión artístico, donde artistas pueden construir esculturas que son a su vez programas computables.

Las sesiones contaron con 1 a 3 facilitadores cuya labor consistió en guiar las presentaciones, resolver dudas y supervisar el progreso. Cada sesión siguió cuatro fases. (1) Una *introducción* de 30 a 45 minutos dirigida por los facilitadores, adaptada al perfil del grupo: más detallada sobre los componentes físicos para los novatos y más técnica para los expertos. (2) Una fase de *exploración libre* de una hora, sin tareas asignadas, en la que los participantes manipulan las piezas a su ritmo para familiarizarse con sus formas y conexiones (3) Una fase de *tareas computacionales* de dos horas, con conjuntos de problemas calibrados al nivel de cada grupo (e.g., sumas iteradas, condicionales para romper bucles); los facilitadores resolvían dudas pero el participante completaba la tarea por su cuenta. (4) Una *encuesta y entrevista* final opcional sobre usabilidad, accesibilidad y experiencia general; en el caso de menores de edad, la encuesta también la respondieron los docentes acompañantes, con preguntas adicionales sobre la viabilidad de SCuLPT como herramienta pedagógica.

El análisis combinó métodos cualitativos y cuantitativos. En lo cuantitativo se midió el tiempo a tarea y el número de errores cometidos durante la fase de programación. En lo cualitativo se analizaron las entrevistas y las preguntas abiertas de la encuesta, buscando patrones recurrentes en la percepción de las piezas, en los obstáculos sintácticos y en la experiencia estética de manipular el lenguaje. Adicionalmente, con el subgrupo de artistas se realizaron sesiones extra dedicadas exclusivamente a la creación de piezas no funcionales, con el fin de evaluar a SCuLPT como medio expresivo independiente de su capacidad computacional.

5.2. Validación empírica

5.2.1. Usabilidad

La fisicalidad de las piezas fue el factor más recurrente en los reportes positivos, especialmente en los grupos jóvenes, que identificaron y diferenciaron los componentes con facilidad. La integridad material de las piezas fue un punto de atención: el PLA es un plástico relativamente blando, propenso a deformación con uso repetido. Algunas piezas se perdieron a lo largo de las sesiones y otras mostraron desgaste, pero el conjunto resistió el ciclo de uso sin afectar la continuidad de las pruebas.

5.2.2. Programación

Aproximadamente el 50 % del total de participantes completó las tareas computacionales asignadas. El desempeño por sub-población, sin embargo, no es el esperable. La Tabla 4 resume los hallazgos por grupo.

Tabla 4: Hallazgos de las tareas computacionales por sub-población.

Sub-población	Complejidad	Observación cualitativa
Niños	Baja	Excelente manejo físico; dificultad conceptual.
Adolescentes novatos	Media	Completaron en $\sim 1,5$ h promedio.
Adultos novatos	Baja	Comprensión del lenguaje sin completar tareas.
Programadores expertos	Baja	Dificultad similar a novatos.
Artistas	Nula	Comprenden las piezas, no completan tareas.

El resultado más inesperado fue la dificultad de los programadores expertos: pese a su experiencia previa, no superaron a los adolescentes novatos en completitud. Una lectura plausible es que el modelo mental que aporta la experiencia con lenguajes textuales no transfiere directamente

al razonamiento sobre pilas LIFO ensambladas físicamente. Los adolescentes, sin esos modelos previos, abordaron el lenguaje en sus propios términos.

5.2.3. Datos de la encuesta

La encuesta cuantitativa (escala Likert 1–5) se aplicó a un sub-conjunto de $n = 25$ participantes de forma voluntaria para los que se obtuvieron respuestas completas. Las preguntas se agruparon en cinco bloques: (1) información demográfica y de experiencia previa, (2) aspectos cuantitativos del lenguaje (e.g., dificultad, claridad), (3) aspectos cualitativos del lenguaje (e.g., disfrute, creatividad, percepción de utilidad), (4) evaluación comparativa de conocimientos de programación, y (5) percepción de docentes/evaluadores sobre el potencial pedagógico de SCuLPT. La Tabla 5 resume los rangos de media por bloque de preguntas.

Tabla 5: Agregados de la encuesta cuantitativa ($n = 25$, escala 1–5).

Bloque de preguntas	Media (rango)
Aspectos cuantitativos del lenguaje	2,2 – 3,4
Evaluación comparativa frente a lenguajes textuales	2,8 – 3,5
Percepción de docentes/evaluadores	4,0 – 4,3

La diferencia entre los bloques es informativa. La percepción técnica de los participantes es media-baja, consistente con la dificultad reportada para completar las tareas. La percepción de los docentes que acompañaron las sesiones es claramente positiva, lo que sugiere viabilidad pedagógica incluso cuando el desempeño inmediato de los aprendices es modesto. Las respuestas cualitativas refuerzan ambos lados: los participantes destacan que “llevar programación al mundo 3D es una hazaña” y que el lenguaje es “interesante y creativo para niños pequeños”, pero también reportan que “los bloques a veces generan confusión” y que “los conceptos de operaciones son confusos”. Las sugerencias más frecuentes apuntan a material didáctico de apoyo (diapositivas, ejercicios incrementales) y a integración con dominios concretos como videojuegos o matemáticas.

5.2.4. Estética

Las sesiones con artistas dieron los resultados más positivos en percepción. Los participantes valoraron la manipulación tangible y reportaron que “la fisicalidad fue quintaesencial a su experiencia”. Aunque no completaron las tareas asignadas, expresaron satisfacción con el proceso e interés por seguir usando el lenguaje, al punto de solicitar sesiones adicionales en las que construyeron piezas no funcionales con propósito puramente estético. El uso emergente que dieron a las piezas, como ensambles no previstos en la especificación, exploraciones de la forma por la forma, y colaboraciones entre escultura e ilustración valida la apertura estética como característica diferenciadora del lenguaje.

6. Discusión

Los resultados de la Sección 5 dibujan una imagen ambivalente que merece discutirse con cuidado. Por un lado, SCuLPT es técnicamente exitoso: el lenguaje es Turing-completo, su intérprete funciona, las piezas físicas resisten el uso y la mayoría de los participantes identifica y manipula los componentes sin dificultad. Por otro, escribir programas en SCuLPT sigue siendo difícil. Aproximadamente la mitad de los participantes completa las tareas computacionales, y los programadores expertos no superan a los adolescentes novatos en esa métrica.

Esta tensión no es un fracaso del lenguaje puntualmente, sino es una de sus características constitutivas. SCuLPT no busca eliminar la dificultad intrínseca de programar, ese problema lo comparten todos los lenguajes. La intención cambiar la *naturaleza* de esa dificultad y, al hacerlo,

ampliar quiénes la encuentran interesante. Lo que los resultados muestran es que la dificultad se reasigna: cae con fuerza la barrera de acceso a la primera interacción con el lenguaje, los niños manipulan piezas y los artistas las disfrutan. Se conserva, casi intacta, la dificultad de razonar algorítmicamente sobre pilas, condicionales y bucles. La discrepancia entre la percepción técnica baja de los participantes (medias Likert 2,2–3,4) y la percepción docente alta (4,0–4,3) refuerza esta lectura. SCuLPT opera mejor como vehículo didáctico mediado por un facilitador que como herramienta de programación autónoma.

La fisicalidad y estética merece tratarse como objeto de estudio en sí misma, no solo como medio. Las sesiones con artistas son el caso más claro de esto. Participantes que reportaron mayor satisfacción estética son también los que más alejaron las piezas de su uso computacional prescrito, construyendo esculturas no funcionales y explorando ensambles no previstos en la especificación. El lenguaje, en sus manos, dejó de ser una herramienta para volverse un material. Esta observación conecta directamente con preguntas abiertas en la comunidad de *programming experience*: qué se siente programar en un medio que se puede sostener, qué propósitos creativos emergen cuando el código deja la pantalla.

Posicionamiento. SCuLPT no aspira a reemplazar los lenguajes textuales. Su aparatosa naturaleza para proyectos de software de gran envergadura es instantáneamente aparente. El tamaño de un programa está acotado por la cantidad de piezas que físicamente se pueden ensamblar, el costo de impresión crece linealmente con esa cantidad, el ensamblaje toma tiempo y, al ser solo físico, no se corre el resultado ensamblado en un computador. SCuLPT funciona, en cambio, como puerta de entrada para audiencias históricamente alejadas de la programación y como medio de expresión para artistas que quieren incorporar cómputo en su práctica sin pasar por el teclado.

Limitaciones. Además de las restricciones de tamaño ya mencionadas, conviene nombrar tres límites más. La primera y más grande obedece a la naturaleza física del ejercicio. Las sesiones de validación requieren de facilitadores que guían a los participantes, resuelven dudas, arman esculturas, supervisan el uso de las piezas y, en el caso de los menores de edad, se aseguran de que los docentes acompañen a los estudiantes. Esto hace que la escalabilidad del ejercicio sea limitada, y que su aplicación en contextos educativos dependa de la disponibilidad de facilitadores capacitados. La segunda limitación va de la mano con la primera. El uso de piezas impresas en 3D con acoples magnéticos es una solución práctica para prototipar y manufacturar el lenguaje a un costo accesible. Sin embargo, el material es frágil y se desgasta con el uso, reduciendo el inventario de cada sesión. Adicionalmente, el costo temporal de imprimir piezas suficientes para dictar los talleres es elevado con una disponibilidad limitada de impresoras 3D. Esto limita mucho la cantidad de participantes que se pueden atender en una sesión. Esto es evidente en los resultados: el número de participantes es bajo, y el número de tareas computacionales asignadas es limitado. Esta limitación se puede superar con materiales más resistentes, procesos de impresión masivos y optimización del diseño. Tercero, las muestras de validación, aunque diversas, no son grandes: el desempeño educativo a más largo plazo, en cohortes estables, sigue siendo una pregunta abierta.

7. Conclusiones y trabajo futuro

Este artículo presenta SCuLPT, un lenguaje de programación tangible cuyas piezas se imprimen en 3D y se ensamblan físicamente para construir esculturas Turing-completas. Sobre un conjunto mínimo de 13 instrucciones que operan sobre pilas LIFO, se demostró formalmente la completitud del lenguaje mediante una traducción Φ que mapea máquinas de Turing arbitrarias a programas SCuLPT. La libertad estética de las piezas distingue al lenguaje frente a las tres familias de trabajo relacionado y, al mismo tiempo, amplía las posibilidades de accesibilidad y de uso expresivo.

La validación con mostró un patrón consistente. La fisicalidad funciona como puerta de entrada y los componentes son reconocibles para audiencias diversas, pero programar sigue requiriendo razonar algorítmicamente y eso no se elimina por cambiar el soporte. La percepción docente, más positiva que la de los aprendices, sugiere que el potencial pedagógico de SCuLPT es mayor que el que sus resultados de corto plazo permiten medir. Los artistas, por su parte, encontraron en el lenguaje un medio expresivo en sí mismo, más allá de su capacidad computacional sin dejarla de lado.

Trabajo futuro. Quedan abiertas varias líneas de investigación sobre el lenguaje y sus capacidades artísticas educacionales. Primero, estudios más extensos en contextos escolares y talleres, con cohortes estables que permitan medir el aprendizaje longitudinal y no solo el desempeño en sesión única. Segundo, mejoras a SCuLPTER como herramienta de desarrollo y como guía de referencia. Agregar elementos como un sistema de manejo de errores más informativo, herramientas de apoyo como un *language server* con *syntax highlighting* y completación de comandos, y un visualizador que permita ver el ensamble final antes de imprimir las piezas. Tercero, herramientas dirigidas específicamente a artistas. Herramientas como exportación de los bloques de una escultura como modelos 3D listos para impresión, y un *escáner de esculturas* capaz de leer una construcción física y reconstruir el programa correspondiente por visión por computador. Cuarto, exploración de SCuLPT en contextos artísticos ampliados como instalaciones interactivas, performances, talleres multidisciplinarios.

Esta última línea el lenguaje como objeto de *programming experience*, donde lo que importa no es solo qué se computa sino qué se siente al hacerlo es la que más promete. La fisicalidad y abstracción geométrica de las sentencias de SCuLPT más que una particularidad es el llamado a repensar qué es crear código, quién puede hacerlo y por qué lo haría. El formalismo y descripción lógica de sus piezas es la invitación a replantear que es crear con código, quién puede hacerlo y por qué lo haría. El código como arte, el arte como código.

Referencias

- [1] Samuel Aaron, Alan F. Blackwell y Pamela Burnard. “The development of Sonic Pi and its use in educational partnerships: Co-creating pedagogies for learning computer programming”. En: *Journal of Music, Technology and Education* 9.1 (2016), págs. 75-94. ISSN: 1752-7074. DOI: 10.1386/jmte.9.1.75_1.
- [2] Andrew Begel. “LogoBlocks: A graphical programming language for interacting with the world”. En: *Electrical Engineering and Computer Science Department, MIT*. Cambridge, MA, 1996.
- [3] Yorah Bosse y Marco Aurélio Gerosa. “Why is programming so difficult to learn? Patterns of difficulties related to programming learning mid-stage”. En: *ACM SIGSOFT Software Engineering Notes* 41.6 (2017), págs. 1-6. ISSN: 0163-5948. DOI: 10.1145/3011286.3011301.
- [4] Nick Collins, Alex McLean, Julian Rohrerhuber y Adrian Ward. “Live coding in laptop performance”. En: *Organised Sound*. Vol. 8. 3. 2003, págs. 321-330. DOI: 10.1017/S135577180300030X.
- [5] Neil Fraser. “Ten things we’ve learned from Blockly”. En: *Proceedings of the IEEE Blocks and Beyond Workshop*. 2015, págs. 49-50. DOI: 10.1109/BLOCKS.2015.7369000.
- [6] Brian Harvey, Daniel D. Garcia, Tiffany Barnes, Nathaniel Titterton, Daniel Armendariz, Luke Segars, Eugene Lemon, Sean Morris y Josh Paley. “Snap! (Build Your Own Blocks)”. En: *Proceedings of the 44th ACM Technical Symposium on Computer Science Education (SIGCSE)*. 2013, pág. 759. DOI: 10.1145/2445196.2445507.

- [7] Michael S. Horn y Robert J. K. Jacob. “Designing tangible programming languages for classroom use”. En: *Proceedings of the 1st International Conference on Tangible and Embedded Interaction (TEI)*. 2007, págs. 159-162. DOI: 10.1145/1226969.1227003.
- [8] Hiroshi Ishii y Brygg Ullmer. “Tangible bits: towards seamless interfaces between people, bits and atoms”. En: *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems*. 1997, págs. 234-241. DOI: 10.1145/258549.258715.
- [9] Amy J. Ko. “How my broken elbow made the ableism of computer programming personal”. En: *Nature* (sep. de 2023). ISSN: 0028-0836. DOI: 10.1038/d41586-023-02885-y.
- [10] Paul Marshall. “Do tangible interfaces enhance learning?” En: *Proceedings of the 1st International Conference on Tangible and Embedded Interaction (TEI)* (2007), págs. 163-170. DOI: 10.1145/1226969.1227004.
- [11] Maria Montessori. *The Absorbent Mind*. Reprinted 2012. Madras: Theosophical Publishing House, 1949.
- [12] Alannah Oleson, Meron Solomon, Christopher Perdriau y Amy J. Ko. “Teaching inclusive design skills with the CIDER assumption elicitation technique”. En: *ACM Transactions on Computer-Human Interaction* 30.1 (2023), págs. 1-49. ISSN: 1073-0516. DOI: 10.1145/3549074.
- [13] Primo Toys. *Cubetto: hands-on coding for children*. <https://www.primotoys.com/cubetto/>. Accessed 2026-05-12.
- [14] Casey Reas y Ben Fry. *Processing: A Programming Handbook for Visual Designers and Artists*. Cambridge, MA: MIT Press, 2007.
- [15] Mitchel Resnick, John Maloney, Andrés Monroy-Hernández, Natalie Rusk, Evelyn Eastmond, Karen Brennan, Amon Millner, Eric Rosenbaum, Jay Silver, Brian Silverman y Yasmin Kafai. “Scratch: programming for all”. En: *Communications of the ACM* 52.11 (2009), págs. 60-67. DOI: 10.1145/1592761.1592779.
- [16] Anthony Robins, Janet Rountree y Nathan Rountree. “Learning and teaching programming: a review and discussion”. En: *Computer Science Education* 13.2 (2003), págs. 137-172. DOI: 10.1076/csed.13.2.137.14200.
- [17] *SCuLPT source code and language specification repository*. <https://github.com/Darkoyd/SCuLPT>. Accessed 2026-05-20.
- [18] *SCuLPTER: web environment and runtime for SCuLPT*. <https://github.com/Darkoyd/SCuLPTER>. Accessed 2026-05-20.
- [19] Amanda Sullivan y Marina Umaschi Bers. “Robotics in the early childhood classroom: experiences with the KIBO robotics kit in preschool classrooms”. En: *International Journal of Technology and Design Education*. Vol. 26. 1. 2016, págs. 3-20. DOI: 10.1007/s10798-015-9304-5.
- [20] Hideyuki Suzuki e Hiroshi Kato. “Interaction-level support for collaborative learning: AlgoBlock—an open programming language”. En: *Proceedings of the First International Conference on Computer Support for Collaborative Learning (CSCL)*. 1995, págs. 349-355. DOI: 10.3115/222020.222828.
- [21] Tangible Play. *Osmo Coding: physical-digital coding games for children*. <https://www.playosmo.com/en/coding/>. Accessed 2026-05-12.
- [22] Hongji Yang y Lu Zhang. “Promoting creative computing: origin, scope, research and applications”. En: *Digital Communications and Networks* 2.2 (2016), págs. 84-91. ISSN: 2352-8648. DOI: 10.1016/j.dcan.2016.02.001.